



VILNIUS UNIVERSITY

ŠIAULIAI ACADEMY

BACHELOR PROGRAMME SOFTWARE ENGINEERING

VILNIUS UNIVERSITY – ŠIAULIAI ACADEMY

Course: Object-Oriented Programming

Student: Soufiane Ouab

Date: April 5, 2026

UML Class Diagram – Job Inheritance

1. Introduction

This report presents a UML (Unified Modeling Language) Class Diagram designed to illustrate the concept of job inheritance in an object-oriented programming context. The diagram consists of three classes: one superclass and two subclasses, connected through a generalization (inheritance) relationship as defined in UML standards.

The purpose of this assignment is to demonstrate an understanding of UML notation, class diagram structure, and the principles of inheritance in object-oriented design.

2. UML Class Diagram Theory

A UML Class Diagram is a structural diagram used in software engineering to represent the static structure of a system. It depicts classes, their attributes, methods, and the relationships between objects.

Class Representation: Each class is drawn as a rectangle divided into three sections:

1. Class Name – at the top, bold or centered

2. Attributes – listed in the middle section
3. Methods – listed in the bottom section

Access Modifiers:

Symbol	Meaning
+	Public – accessible from anywhere
-	Private – accessible only within the class
#	Protected – accessible within the class and subclasses
~	Package – accessible within the same package

Multiplicity defines how many objects participate in a relationship:

Notation	Meaning
1	Exactly one instance
0..1	Zero or one instance
*	Many instances
1..*	One or more instances

Association — a basic relationship between two classes. Shown as a simple line connecting them. Example: A Driver uses a Car.

Aggregation — a whole-part relationship where the part can exist independently. Shown with a hollow diamond (◇). Example: A Library contains Books, but books can exist without the library.

Composition — a strong whole-part relationship where the part cannot exist without the whole. Shown with a filled diamond (◆). Example: A House has Rooms — if the house is deleted, rooms cease to exist.

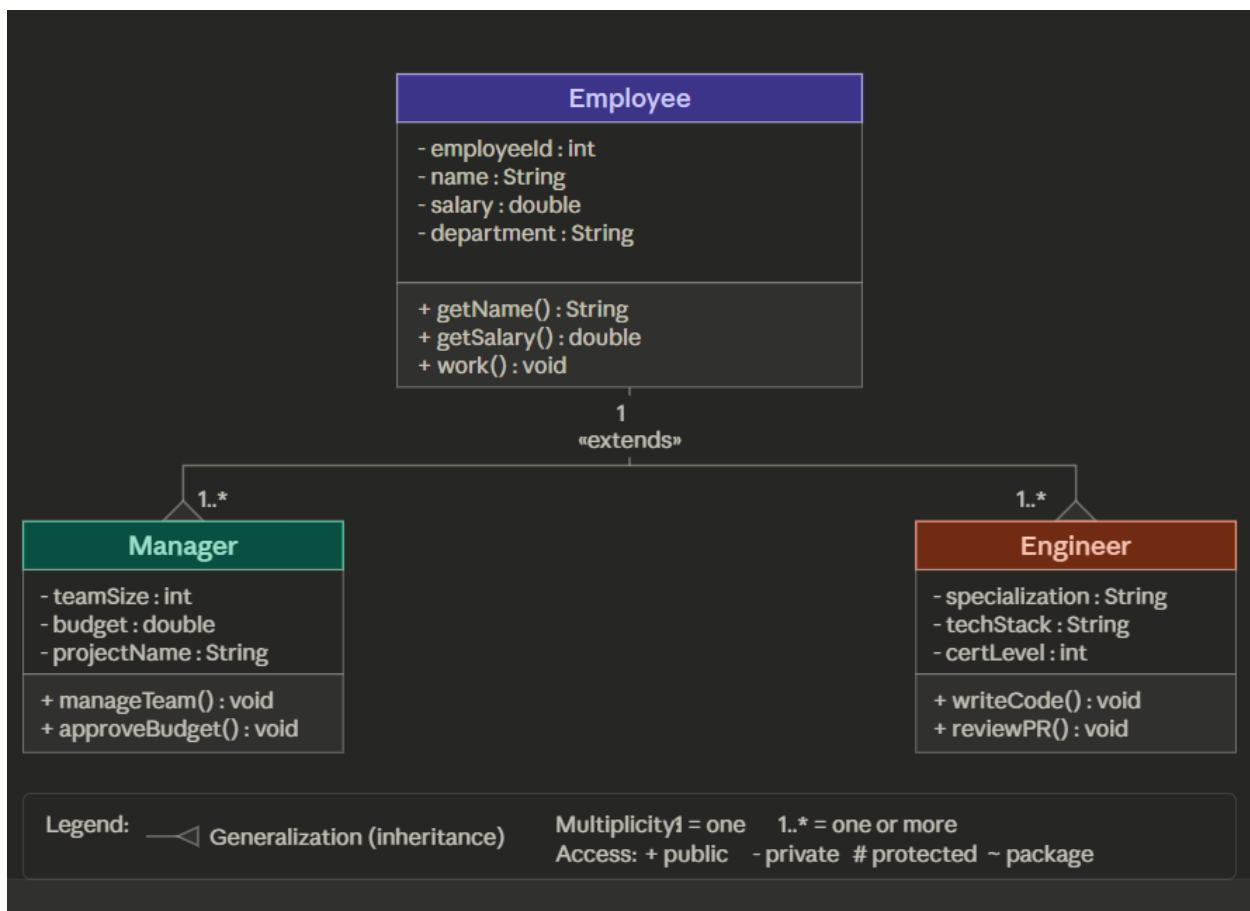
Dependency — one class depends on another but does not own it. Shown as a dashed arrow (- - ->). Example: A Car depends on Fuel.

Summary of all relationships:

Type	Notation	Meaning
Association	—	Basic link between classes
Aggregation	◇—	Weak whole-part
Composition	◆—	Strong whole-part
Generalization	—△	Inheritance / is-a
Dependency	- - →	One class uses another

Generalization (Inheritance): Generalization represents an "is-a" relationship between a superclass and a subclass. It is shown with a solid line and an open triangle (△) pointing toward the superclass. The subclass inherits all attributes and methods of the superclass and may add its own.

3. UML Class Diagram – Job Inheritance



4. Class Descriptions

4.1 Employee (Superclass)

The Employee class is the parent class and contains shared attributes and behaviors common to all types of employees:

- employeeId – a unique identifier for each employee
- name – the full name of the employee
- salary – the employee's compensation
- department – the organizational unit the employee belongs to
- getName() – returns the employee's name
- getSalary() – returns the salary value
- work() – a general method inherited and used by all subclasses

4.2 Manager (Subclass)

The Manager class extends Employee and represents employees with leadership and managerial responsibilities. It adds:

- teamSize – the number of team members under the manager
- budget – the financial budget the manager controls
- projectName – the name of the current assigned project
- manageTeam() – handles team coordination and leadership
- approveBudget() – approves or manages financial budgets

4.3 Engineer (Subclass)

The Engineer class extends Employee and represents employees with technical responsibilities. It adds:

- specialization – the area of engineering (e.g., software, civil, mechanical)
- techStack – the technologies and tools the engineer uses
- certLevel – the certification or skill level of the engineer
- writeCode() – develops and writes software

- reviewPR() – reviews pull requests and technical work

5. Relationships Summary

From	To	Relationship Type	Description
Manager	Employee	Generalization (Inheritance)	Manager is-a Employee
Engineer	Employee	Generalization (Inheritance)	Engineer is-a Employee

6. Conclusion

This assignment demonstrated the creation of a 3-class UML Class Diagram based on job inheritance. The Employee superclass was designed to encapsulate shared properties, while the Manager and Engineer subclasses extended it with role-specific attributes and behaviors.

This approach reflects core object-oriented design principles such as reusability, modularity, and the open/closed principle. UML diagrams serve as a universal language in software engineering for communicating system structure clearly and professionally.